SHOP'IMT EC2 : LLD

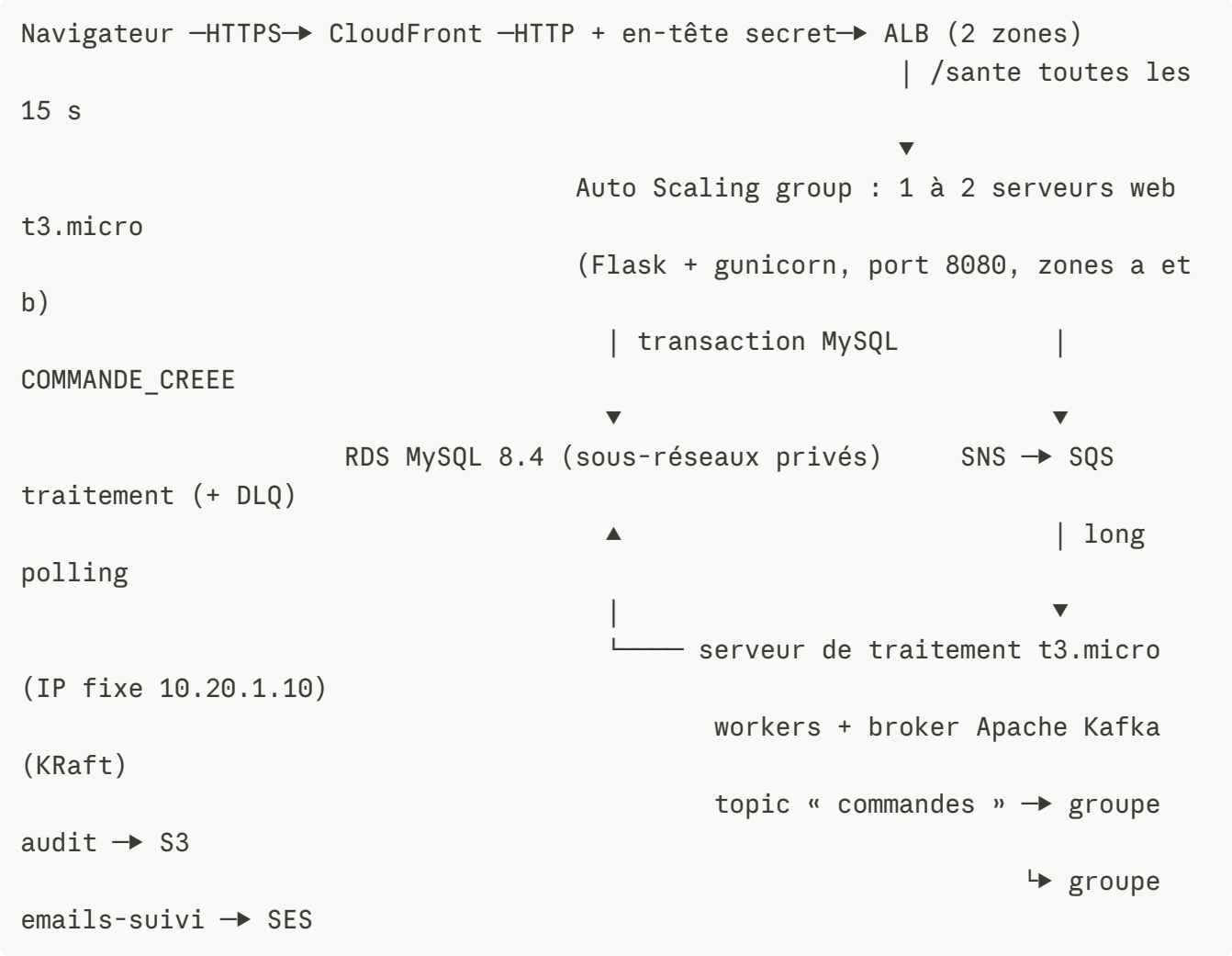
Oct 30, 2026 · @Louison

Objet et périmètre

Ce document décrit la **solution 2 (machines virtuelles EC2)** de SHOP'IMT. Elle tourne **en même temps** que la solution serverless, sur le même compte, avec le même front, la même API et les mêmes e-mails : seule l'infrastructure change. On peut ainsi passer une commande sur chacune et comparer.

- **Code** : `infra/modules/ec2` (Terraform) et `backend/ec2` (serveurs Python), dans le dépôt https://github.com/louizoom/shop-imt.
- **Activation** : clé `solution_ec2` du paramètre SSM `/shopimt/<env>/config`. Absente ou `false` : rien n'est créé. Le serverless n'est jamais touché.
- **Réutilisation** : les serveurs importent les modules des Lambdas (validation des commandes, numéros `SHOP-000042-K7QF`, date de livraison, agenda `.ics`, tirage du plan d'expédition, e-mails HTML FR/EN). Une règle métier ne s'écrit qu'une fois.

Vue d'ensemble de l'architecture



Service	Rôle dans la solution EC2
VPC	Réseau dédié `10.20.0.0/16` sur 2 zones : sous-réseaux publics (ALB, serveurs) et privés (RDS, sans route Internet)
Internet Gateway, endpoint Gateway S3	Sortie vers les API AWS sans NAT Gateway ; le trafic S3 reste sur le réseau AWS (gratuit)
EC2 (t3.micro, Amazon Linux 2023)	Serveurs web et serveur de traitement, IMDSv2 obligatoire, disques chiffrés
Auto Scaling	1 à 2 serveurs web répartis sur 2 zones, ajout au-delà de 60 % de CPU, remplacement d'une instance malade, mise à jour progressive
Elastic Load Balancing (ALB)	Répartition de charge, contrôle de santé `/sante`, accepte seulement CloudFront

Service	Rôle dans la solution EC2
CloudFront	HTTPS, cache des fichiers statiques, protection de l'ALB
RDS for MySQL 8.4	Catalogue, commandes, historique ; TLS obligatoire, chiffrement KMS, sauvegarde quotidienne
SNS, SQS	Diffusion de `COMMANDE_CREEE` vers la file de traitement, DLQ après 3 échecs
Apache Kafka sur EC2	Flux de tout le cycle de vie des commandes, deux groupes de consommateurs
SES	E-mails de confirmation et de suivi (même domaine vérifié que le serverless)
SSM	Mot de passe RDS en SecureString, Session Manager à la place de SSH, AMI la plus récente
CloudWatch	Agent (logs, mémoire), alarmes, récupération automatique, tableau de bord
IAM	Un rôle par type de serveur (profil d'instance), aucune clé d'accès

Parcours d'une commande

1. **Page et catalogue.** Le navigateur ouvre l'adresse CloudFront. Les fichiers `/assets/*` et `/img/*` sont servis depuis le cache de CloudFront ; le reste passe par l'ALB jusqu'à un serveur web, qui lit `produits` dans RDS.
2. **Commande.** `POST /commandes` arrive sur l'un des serveurs web. La validation est celle de la Lambda `commandes`.
3. **Transaction MySQL.** `SELECT … FOR UPDATE` verrouille les lignes des produits commandés : deux commandes simultanées, même sur deux serveurs différents, s'attendent. Le stock est vérifié puis décrémenté, la commande et sa première ligne d'historique sont écrites, et le tout est validé en une fois (`COMMIT`). Stock insuffisant : `ROLLBACK` et réponse 409.
4. **Numéro.** La clé `AUTO_INCREMENT` de MySQL donne le numéro `SHOP-000042`, suivi de 4 caractères aléatoires (même format que le compteur DynamoDB du serverless).
5. **Événements.** Le serveur web publie `COMMANDE_CREEE` sur **SNS** (vers la file SQS de traitement) et sur **Kafka** (topic `commandes`, clé = numéro de commande).

6. **Traitement.** Le serveur de traitement lit la file SQS en long polling. Il réserve la commande (verrou de ligne, une seule fois), envoie l'e-mail de confirmation SES, puis joue les étapes tirées au sort : `EN_PREPARATION`, `EMBALLEE`, `EXPEDIEE`, parfois `INCIDENT_TRANSPORT`, `EN_LIVRAISON`, `LIVREE`.
7. **Statuts.** Chaque étape est écrite dans RDS (statut et historique) et publiée sur Kafka.
8. **Audit.** Le groupe Kafka `audit` écrit chaque événement dans S3 : `audit/AAAA/MM/JJ/<commande>/<partition>-<offset>-<statut>.json`.
9. **E-mails de suivi.** Le groupe Kafka `emails-suivi` envoie les e-mails d'expédition, d'incident et de livraison.
10. **Suivi.** La page `/suivi/<numéro>` interroge `GET /commandes/<numéro>` toutes les 2 secondes jusqu'à `LIVREE`.

Conception détaillée

Réseau

Sous-réseau	Zone	Plage	Contenu
public a	eu-west-3a	10.20.1.0/24	ALB, serveurs web, serveur de traitement (10.20.1.10)
public b	eu-west-3b	10.20.2.0/24	ALB, serveurs web
privé a	eu-west-3a	10.20.11.0/24	RDS
privé b	eu-west-3b	10.20.12.0/24	RDS (2e zone exigée par RDS, utilisée par Multi-AZ)

Les groupes de sécurité forment une chaîne : chaque couche n'accepte que la précédente.

Groupe	Entrée autorisée	Sortie autorisée
alb	80 depuis la liste de préfixes CloudFront	8080 vers web
web	8080 depuis alb	443 et 80 (API AWS, paquets), 3306 vers bdd, 9092 vers traitement
traitement	9092 depuis web	443 et 80, 3306 vers bdd

Groupe	Entrée autorisée	Sortie autorisée
bdd	3306 depuis web et traitement	aucune

Aucun port SSH n'est ouvert : l'accès aux serveurs passe par **SSM Session Manager**, journalisé, sans clé SSH.

CloudFront et ALB

- CloudFront ajoute un en-tête secret (`X-Origine-Shopimt` , valeur aléatoire générée par Terraform). L'ALB transmet aux serveurs seulement les requêtes qui le portent ; les autres reçoivent 403. Avec la liste de préfixes CloudFront dans le groupe de sécurité, l'ALB est inaccessible en direct.
- Cache : `CachingOptimized` pour `/assets/*` (noms avec empreinte Vite) et `/img/*` , `CachingDisabled` pour le reste (API et pages).
- Groupe cible : contrôle `/sante` toutes les 15 s, 2 succès pour entrer, 3 échecs pour sortir. `/sante` exécute `SELECT 1` sur RDS : un serveur qui n'atteint plus la base est retiré du trafic.

Auto Scaling

- Modèle de lancement : dernière AMI Amazon Linux 2023 (paramètre public SSM), t3.micro, IMDSv2 obligatoire, disque gp3 8 Go chiffré, crédits CPU `standard` (aucun surcoût).
- Groupe : minimum 1, maximum 2, sur les sous-réseaux des deux zones, contrôle de santé **ELB** (une instance jugée malade par l'ALB est remplacée).
- Politique de suivi de cible : CPU moyen 60 %. Au-delà, une 2e instance démarre ; quand la charge retombe, elle est supprimée.
- **Instance refresh** : chaque nouvelle version du code (empreinte du zip dans le nom de l'objet S3) crée une version du modèle et remplace les instances une par une, la nouvelle étant saine avant l'arrêt de l'ancienne.

Serveur de traitement et Kafka

- Instance unique à IP privée fixe, pour que les serveurs web trouvent le broker sans service de découverte.
- **Récupération automatique EC2** : une alarme CloudWatch sur `StatusCheckFailed_System` relance l'instance sur un autre hôte physique, avec la même IP et le même disque.

- Kafka 3.9.1 en mode **KRaft** (un nœud broker et contrôleur, sans ZooKeeper), mémoire limitée à 320 Mo, 1 Go de swap. L'archive de Kafka est mise en cache dans S3 au premier démarrage.
- Topic `commandes`, 3 partitions. La clé est le numéro de commande : tous les événements d'une commande restent dans l'ordre dans la même partition.
- Deux groupes de consommateurs lisent tout le flux, chacun avec ses propres offsets, validés après traitement (au moins une fois).
- Si Kafka est arrêté, une commande est quand même acceptée et traitée (SNS et SQS suffisent) ; l'envoi Kafka abandonne au bout de 2 s et le journal indique `KAFKA_INDISPONIBLE`.
- Un balayage RDS toutes les minutes rattrape une commande restée `RECUE` plus de 2 minutes (par exemple si SNS a échoué juste après le `COMMIT`).

Base RDS

Table	Colonnes principales	Rôle
produits	id, nom, nom_en, prix, stock (contrainte `stock >= 0`)	Catalogue, créé au premier démarrage
commandes	numero (AUTO_INCREMENT), order_id, client_email, langue, total, statut, articles (JSON), transporteur, numero_suivi, cree_le, livraison_estimee	Commandes
historique	order_id, horodatage, statut, détail	Une ligne par étape

- MySQL 8.4 (version LTS ; 8.0 est passé en support étendu payant).
- `require_secure_transport = 1` : connexion TLS obligatoire, les serveurs vérifient le certificat RDS.
- Mot de passe de 32 caractères généré par Terraform, rangé dans SSM Parameter Store (SecureString, clé KMS gérée `aws/ssm`). Les serveurs le lisent au démarrage avec leur rôle IAM ; il n'apparaît ni dans le code ni dans le user_data.
- Sauvegarde automatique quotidienne, restauration à la seconde près sur 24 h. Stockage chiffré, 20 Go sans extension automatique.

Démarrage d'un serveur (user_data)

1. Swap de 1 Go, installation de Python 3.12 et de l'agent CloudWatch (et de Java 17 sur le serveur de traitement).

2. Téléchargement du code (et du front pour les serveurs web) depuis S3, par l'endpoint Gateway du VPC.
3. Bundle de certificats RDS, environnement virtuel Python et dépendances (`flask` , `gunicorn` , `pymysql` , `kafka-python` , `boto3`).
4. Service systemd `shopimt` démarré tout de suite : les commandes sont traitées même avant que Kafka soit prêt.
5. Sur le serveur de traitement : Kafka (depuis le cache S3), formatage du stockage KRaft, service `kafka` , création du topic.
6. Agent CloudWatch (journaux et mémoire).

Rôles IAM

Rôle	Droits propres	Communs
web	`sns:Publish` sur le topic	lecture de `code/*` dans S3, lecture du paramètre du mot de passe, Session Manager, agent CloudWatch
traitement	lecture et suppression dans la file SQS, `s3:PutObject` sur `audit/*` et `kafka/*` , envoi SES	idem

Haute disponibilité et résilience

Panne	Ce qui se passe
Un serveur web tombe	L'ALB le retire en 45 s au plus, l'Auto Scaling en lance un autre (dans l'autre zone si besoin)
Une zone de disponibilité tombe	L'ALB n'envoie plus de trafic vers elle, l'Auto Scaling relance dans l'autre zone
Pic de charge	2e serveur web au-delà de 60 % de CPU
Serveur de traitement arrêté	Les commandes sont acceptées et attendent dans SQS (14 jours), puis sont traitées au redémarrage
Panne matérielle du serveur de traitement	Récupération automatique EC2, même IP et même disque

Panne	Ce qui se passe
Broker Kafka arrêté	Commandes et e-mails de confirmation continuent ; seuls les événements Kafka de cette période manquent
RDS	Mono-AZ (gratuit) : la sauvegarde permet la restauration. En production, Multi-AZ (variable `bdd_multi_az`) bascule en 1 à 2 minutes sur l'instance de secours

Coût

Le compte est encore dans l'offre gratuite de 12 mois (750 h par mois d'EC2 t3.micro, de RDS db.t4g.micro, d'ALB et d'adresse IPv4 publique).

Élément	Gratuit	Au-delà
2 instances t3.micro 24 h/24	750 h sur 1 440 h	~0,27 $/jour
RDS db.t4g.micro, 20 Go	750 h et 20 Go	0
ALB	750 h et 15 LCU	0
4 IPv4 publiques (2 ALB, 2 instances)	750 h sur 2 880 h	~0,35 $/jour
CloudFront, SNS, SQS, SES, S3, CloudWatch	offres permanentes	0

Total : environ **0,60 $ par jour** quand la solution EC2 tourne, rien quand elle est désactivée. Volontairement exclus : NAT Gateway (~1 $/jour), Amazon MSK (~2 $/jour pour un cluster minimal), RDS Multi-AZ, 2e serveur web permanent. Ce seraient les choix d'une vraie production.

Comparaison des deux solutions

	Serverless	EC2
Calcul	Lambda (à la demande)	Serveurs t3.micro allumés en permanence
Base	DynamoDB, transaction sur le stock	RDS MySQL, `SELECT … FOR UPDATE`

	Serverless	EC2
Montée en charge	Automatique, sans réglage	Auto Scaling sur le CPU, 1 à 2 serveurs
Haute disponibilité	Services multi-zones gérés par AWS	ALB et Auto Scaling sur 2 zones, à configurer
Événements	SNS, SQS, DynamoDB Streams	SNS, SQS, Kafka
Expédition simulée	Step Functions	Fils d'exécution du serveur de traitement
Coût au repos	0	~0,60 $/jour
Exploitation	Rien à patcher	AMI, paquets, disque, mémoire à surveiller

Correspondance avec le sujet

Exigence	Réponse dans la solution EC2
Solution sur machines virtuelles	EC2 + Auto Scaling + ALB
Catalogue via une interface Web	Même front Vue, servi par les serveurs web
Commander un ou plusieurs produits	`POST /commandes`, transaction MySQL
Confirmation en temps réel	Réponse immédiate, e-mail SES en quelques secondes, page de suivi
Audit et reporting	Groupe Kafka `audit` vers S3, historique dans RDS
Emballage et expédition simulés	Serveur de traitement, mêmes étapes que le serverless
Modèle événementiel SNS	Topic SNS vers file SQS
Streaming Kafka	Broker Kafka sur EC2, deux groupes de consommateurs (MSK présenté comme cible)
Terraform	Module `ec2`, activé par une clé SSM
Haute disponibilité	2 zones, contrôle de santé, remplacement et récupération automatiques
Free Tier	Instances, RDS et ALB dans l'offre gratuite ; surcoût chiffré ci-dessus

Déployer, tester, supprimer

1. Mettre `"solution_ec2": true` dans le paramètre SSM `/shopimt/<env>/config`.
2. Déployer comme d'habitude (fusion sur `dev` ou `main`, ou `terraform apply` dans `infra/envs/<env>`). RDS prend une dizaine de minutes, puis les serveurs environ 5 minutes pour s'installer.
3. L'adresse est dans la sortie `ec2_site_url`. Passer une commande, suivre la page de suivi, ouvrir l'e-mail.
4. Dans la console : tableau de bord CloudWatch `shopimt-ec2`, groupe cible de l'ALB, fichiers `audit/` du bucket `shopimt-ec2-<compte>`, instances dans Session Manager.
5. Test de résilience : arrêter le serveur web dans la console EC2. L'ALB le retire, l'Auto Scaling en relance un, le site reste servi dès que le nouveau est sain.
6. Tests automatiques : `pytest` lance les tests de la solution EC2 contre MySQL (variable `SHOPIMT_TEST_BDD`, service MySQL 8.4 dans la CI).
7. Supprimer : `"solution_ec2": false` puis déploiement. Tout ce qui est propre à la solution EC2 est détruit, le serverless reste en ligne.